Show me a Real DevOps SSH multi-account setup step-by-step lab

Below is a **real DevOps-style SSH multi-account setup lab** exactly like what is used in companies (GitHub personal + company accounts on same machine).

I'll keep it practical so you can actually implement it step by step.

🧪 LAB: Real DevOps SSH Multi-Account Setup (GitHub)

🎯 Goal

You will configure:

- 👤 Personal GitHub account
- 🏢 Work/Company GitHub account
- 🔑 Separate SSH keys for both
- ⚙️ SSH config aliases (no conflicts)
- 🚀 Git operations without confusion

🧱 STEP 1: Check existing SSH keys

```
ls -al ~/.ssh
```

If keys exist like:

- `id_ed25519`
- `id_rsa`

👉 Don't reuse them for multiple accounts in real-world setup.

🔐 STEP 2: Create separate SSH keys

👤 Personal account key

```
ssh-keygen -t ed25519 -C "personal-email@example.com" -f ~/.ssh/id_ed25519_
```

🏢 Work account key

```
ssh-keygen -t ed25519 -C "work-email@company.com" -f ~/.ssh/id_ed25519_wor
```

🚀 STEP 3: Add keys to SSH agent

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519_personal
ssh-add ~/.ssh/id_ed25519_work
```

Check:

```
ssh-add -l
```

⚙️ STEP 4: Configure SSH config file (CORE STEP)

```
nano ~/.ssh/config
```

Paste:

```
# Personal GitHub account
Host github-personal
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_personal
  AddKeysToAgent yes

# Work/Company GitHub account
Host github-work
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_work
  AddKeysToAgent yes
```

🔑 STEP 5: Add SSH keys to GitHub accounts

👤 Personal GitHub

Copy key:

```
cat ~/.ssh/id_ed25519_personal.pub
```

Paste here:

👉 GitHub → Settings → SSH Keys → New Key

🏢 Work GitHub

```
cat ~/.ssh/id_ed25519_work.pub
```

Add to company GitHub account.

🧪 STEP 6: Test SSH connections

Personal:

```
ssh -T git@github-personal
```

Work:

```
ssh -T git@github-work
```

Expected output:

```
Hi username! You've successfully authenticated...
```

📁 STEP 7: Clone repositories correctly

👤 Personal repo

```
git clone git@github-personal:username/repo.git
```

🏢 Work repo

```
git clone git@github-work:company/repo.git
```

🔄 STEP 8: Fix existing repo (IMPORTANT REAL SCENARIO)

If repo already cloned with wrong remote:

```
git remote set-url origin git@github-work:company/repo.git
```

Check:

```
git remote -v
```

⚠️ STEP 9: Avoid common mistakes

× Wrong:

```
git clone git@github.com:company/repo.git
```

✔ Correct:

```
git clone git@github-work:company/repo.git
```

🧠 REAL DEVOPS SCENARIO UNDERSTANDING

🏢 Company laptop situation:

- 1 machine
- 2 GitHub accounts
- multiple projects

Without SSH config:

👉 confusion, wrong commits, credential errors

With SSH config:

👉 clean isolation, zero conflicts, production-safe workflow

🔥 BONUS: Check which key is used

```
ssh -vT git@github-work
```

You'll see:

- which IdentityFile is being used
- debug flow like real DevOps engineers

🚀 Final Summary

You built:

✔ Multi-account SSH system
✔ GitHub personal + work isolation

✔ Production-level DevOps workflow
✔ Alias-based SSH shortcuts

If you want next level upgrade, I can teach you:

🔥 Git + SSH + CI/CD pipeline (GitHub Actions)
🔥 Real company branching strategy (GitFlow)
🔥 AWS server SSH multi-user setup
🔥 Zero-downtime deployment workflow

Just tell me 👍